SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

Private Insight Engine

ADA23CL304 – BIG DATA ANALYTICS LABORATORY

Submitted by

Nitish S M – E0323010
Lokesh Krishna – E0323022
Nithish Kumar - E0323011
Yokesh sharma – E0323020

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(Artificial Intelligence and Data Analytics)

Sri Ramachandra Faculty of Engineering and Technology,
Sri Ramachandra Institute of Higher Education and Research,
Porur, Chennai -600116

June, 2026

SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified to be the Bonafide record of the work done by Nitish, Lokesh krishna, Nithish Kumar**,** Yokesh Sharma of Semester VI, third Year B.Tech Degree course in Sri Ramachandra Faculty of Engineering and Technology, of Computer Science and Engineering Department in the **ADA23CL304 Big Data Analytics Laboratory** during the academic year **2025-2026**.

Signature of Faculty In-charge

Ms.S.Krishnaveni

Assistant Professor,
Department of Artificial Intelligence and Machine Learning,
Sri Ramachandra Faculty of Engineering and Technology,
Sri Ramachandra Institute of Higher Education and Research,
Porur, Chennai, Tamil Nadu.

Submitted for the Big Data Analytics Laboratory project presentation held at Sri Ramachandra Faculty of Engineering and Technology on

Internal Examiner **External Examiner**

TABLE OF CONTENTS

CHAPTER 1

INTRODUCTION

1.1 Overview

The Private Insight Engine is a privacy-focused Retrieval-Augmented Generation (RAG) system designed to enable users to interact with their personal documents using natural language. The system leverages Apache Spark for scalable document processing, Sentence Transformers for generating semantic embeddings, ChromaDB for vector storage, and Ollama for running large language models locally. By combining these technologies, users can query large collections of documents and receive context-aware answers without sending sensitive data to external cloud services.

The system processes documents such as PDFs and text files, converts them into vector representations, stores them in a vector database, and retrieves relevant information based on semantic similarity. The retrieved context is then provided to a local language model to generate accurate and relevant responses.

1.2 Motivation

With the increasing adoption of Artificial Intelligence, many document analysis and question-answering systems rely on cloud-based services, raising concerns regarding data privacy, security, and internet dependency. Organizations and individuals often handle confidential documents that should not be uploaded to external servers.

The motivation behind the Private Insight Engine is to create a secure and efficient AI-powered document assistant that operates entirely on a local machine. The project demonstrates how modern AI technologies, big data processing frameworks, and vector databases can be integrated to provide intelligent document retrieval while maintaining complete user privacy.

1.3 Problem Statement

Traditional document search systems rely primarily on keyword matching, which often fails to understand the semantic meaning of user queries. As the volume of documents increases, finding relevant information becomes time-consuming and inefficient. Additionally, existing AI-powered document assistants frequently require uploading data to cloud services, creating privacy and security concerns.

Therefore, there is a need for a system that can process large collections of local documents, understand the semantic meaning of user queries, retrieve relevant information efficiently, and generate accurate responses while ensuring complete data privacy.

1.4 Objectives

The main objective of this project is to design and implement a privacy-preserving document intelligence system that enables users to interact with their local documents through natural language queries.

The specific objectives are:

- To develop a scalable document ingestion pipeline using Apache Spark.
- To extract and preprocess text from PDF and text documents.
- To generate semantic embeddings using Sentence Transformer models.
- To store document embeddings efficiently using ChromaDB.
- To implement a semantic retrieval mechanism for finding relevant document content.
- To integrate a locally hosted Large Language Model (LLM) through Ollama.
- To generate accurate responses based on retrieved document context.
- To ensure complete privacy by performing all processing locally without cloud dependency.
- To provide an efficient and user-friendly question-answering system for personal document collections.

CHAPTER 2

TECHNIQUES USED

2.1 Apache Spark

Apache Spark is an open-source distributed data processing framework used for handling large-scale data efficiently. In this project, Spark operates in local mode and is responsible for ingesting, processing, and managing document data. Spark DataFrames are used to organize document content into structured formats, enabling efficient text processing and scalability.

Features of Apache Spark

- Fast in-memory data processing.
- Supports large-scale data analytics.
- Provides DataFrame APIs for structured data.
- Scalable and fault-tolerant architecture.

Role in the Project

- Reading PDF and text documents.
- Converting document content into structured DataFrames.
- Performing text preprocessing and chunking operations.

2.2 Text Chunking

Large Language Models (LLMs) have limitations on the amount of text they can process at once. To overcome this limitation, document contents are divided into smaller segments known as chunks.

Process

- Documents are split into chunks of approximately 500 words.
- Overlapping chunks are created to preserve contextual information.
- Each chunk is processed independently for embedding generation.

Benefits

- Improves retrieval accuracy.
- Reduces memory consumption.
- Enhances response quality.

2.3 Sentence Transformers

Sentence Transformers are deep learning models that convert textual information into dense vector representations called embeddings. These embeddings capture semantic meaning rather than simple keyword matches.

Model Used

- all-MiniLM-L6-v2

Advantages

- Lightweight and efficient.
- Generates high-quality semantic embeddings.
- Suitable for retrieval-based applications.

Role in the Project

- Converts document chunks into vector representations.
- Converts user queries into embeddings for similarity comparison.

2.4 Vector Embeddings

Vector embeddings are numerical representations of text in a high-dimensional space. Texts with similar meanings are positioned closer together in this vector space.

Working Principle

- Each chunk is transformed into a vector.
- User queries are transformed into vectors.
- Similarity measures are applied to identify relevant chunks.

Benefits

- Semantic understanding of text.
- More accurate retrieval compared to keyword search.
- Supports intelligent document querying.

2.5 ChromaDB

ChromaDB is an open-source vector database designed specifically for AI applications. It stores embeddings and enables efficient similarity-based retrieval.

Features

- Persistent storage.
- Fast vector search.
- Easy integration with Python applications.
- Optimized for Retrieval-Augmented Generation (RAG) systems.

Role in the Project

- Stores document embeddings.
- Retrieves relevant document chunks based on query similarity.
- Maintains metadata associated with documents.

2.6 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) combines information retrieval techniques with Large Language Models to generate accurate and context-aware responses.

Workflow

1. User submits a query.
2. Query embedding is generated.
3. ChromaDB retrieves relevant document chunks.
4. Retrieved context is sent to the language model.
5. The model generates a response based on the provided context.

Advantages

- Reduces hallucinations.
- Improves answer accuracy.
- Utilizes external knowledge effectively.

2.7 Ollama

Ollama is a framework that enables local execution of Large Language Models without requiring cloud services.

Features

- Offline operation.
- Privacy-preserving architecture.
- Support for multiple open-source LLMs.

- Easy API integration.

Role in the Project

- Hosts the Phi-3 language model locally.
- Generates answers using retrieved document context.
- Ensures complete privacy of user data.

2.8 Semantic Similarity Search

Semantic similarity search identifies documents based on meaning rather than exact keyword matches.

Techniques Used

- Cosine Similarity.
- Vector Distance Measurement.
- Embedding Comparison.

Benefits

- Improved search relevance.
- Better understanding of user intent.
- Enhanced retrieval performance.

2.9 Local AI Processing

The entire system is designed to operate on a local machine without cloud dependency.

Advantages

- Enhanced data privacy.
- Reduced operational cost.
- Offline accessibility.
- Faster access to local documents.

Importance in the Project

All document processing, embedding generation, vector storage, retrieval, and response generation occur locally, ensuring that sensitive information never leaves the user's system.

2.10 System Workflow

The overall workflow of the Private Insight Engine consists of the following stages:

1. Document Ingestion using Apache Spark.
2. Text Extraction and Preprocessing.
3. Text Chunking.
4. Embedding Generation using Sentence Transformers.
5. Storage of Embeddings in ChromaDB.
6. Query Embedding Generation.
7. Semantic Similarity Search.
8. Context Retrieval.
9. Response Generation using Ollama.
10. Presentation of Final Answer to the User.

This workflow enables efficient, scalable, and privacy-preserving document intelligence while maintaining high retrieval accuracy and response quality.

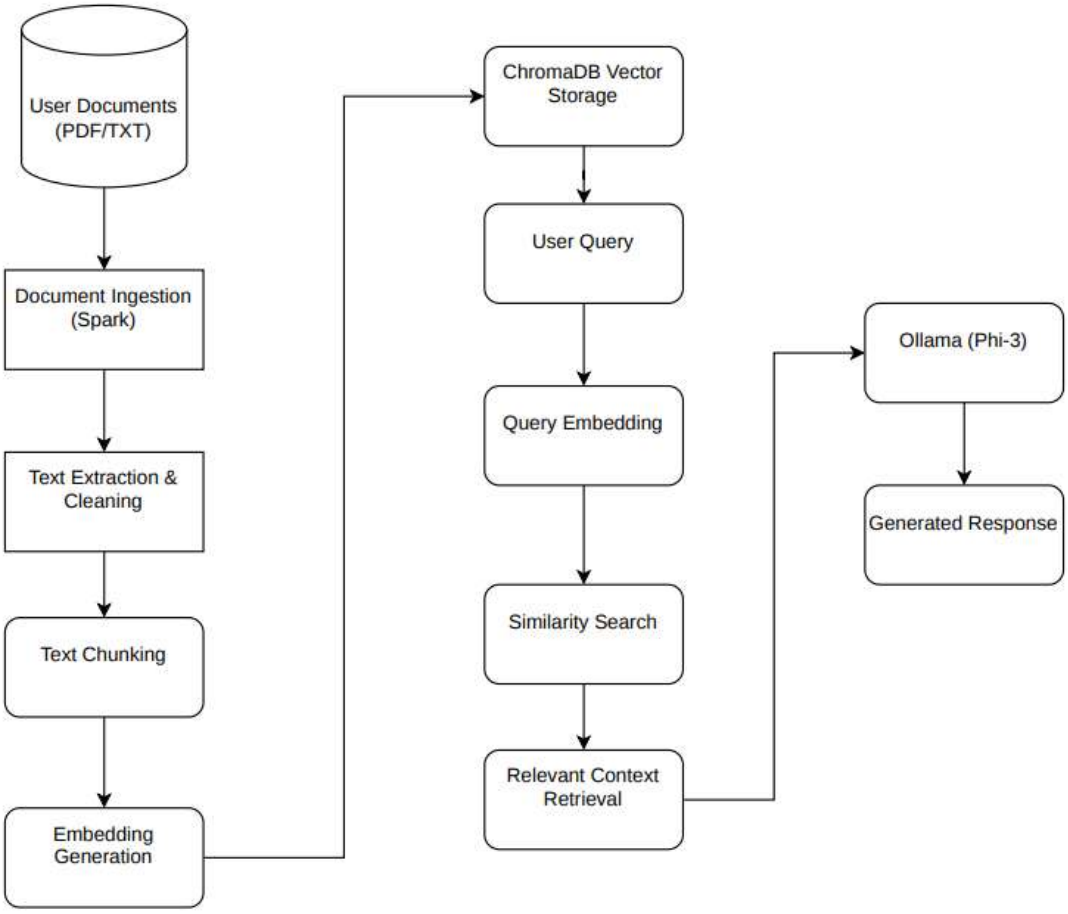
CHAPTER 3

METHODOLOGY

3.1 Introduction

The Private Insight Engine uses a Retrieval-Augmented Generation (RAG) approach to enable users to query local documents using natural language. The system processes documents, generates embeddings, stores them in a vector database, and retrieves relevant information to answer user queries.

3.2 System Workflow



3.3 Document Processing

Documents such as PDF and text files are loaded using Apache Spark. The extracted text is cleaned and divided into smaller chunks to improve processing and retrieval efficiency.

3.4 Embedding Generation

The Sentence Transformer model (all-MiniLM-L6-v2) converts each text chunk into a numerical vector called an embedding. These embeddings represent the semantic meaning of the text.

3.5 Vector Storage and Retrieval

The generated embeddings are stored in ChromaDB. When a user enters a query, the query is converted into an embedding and compared with stored vectors to retrieve the most relevant document chunks.

3.6 Response Generation

The retrieved chunks are provided as context to the Phi-3 model running locally through Ollama. The model generates an answer based on the retrieved information and returns it to the user.

3.7 Summary

The methodology combines Apache Spark, Sentence Transformers, ChromaDB, and Ollama to create a privacy-focused document question-answering system that operates entirely on a local machine.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Implementation

The Private Insight Engine was developed using Apache Spark, Sentence Transformers, ChromaDB, and Ollama. The system processes local documents, generates semantic embeddings, stores them in a vector database, and answers user queries using a locally hosted language model. Modules Implemented

- Document Ingestion
    - Loads PDF and text documents from local storage.
    - Processes document content using Apache Spark.
- Embedding Generation
    - Converts document chunks into semantic vector embeddings using Sentence Transformers.
- Vector Storage
    - Stores embeddings and metadata in ChromaDB for efficient retrieval.
- Query Processing
    - Converts user queries into embeddings and retrieves relevant document chunks.
- Response Generation
    - Uses the Phi-3 model through Ollama to generate context-aware answers.

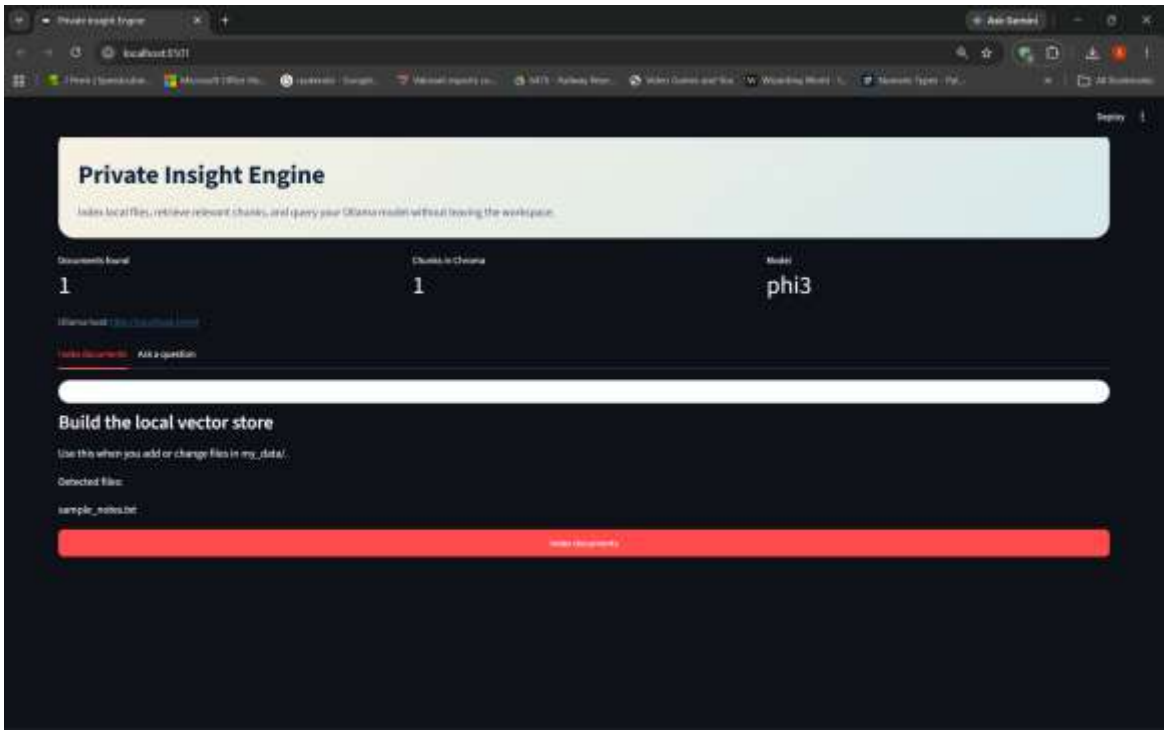


Figure 4.2: Document Indexing and Vector Store Creation

The screenshot displays the document indexing module of the system. Uploaded documents are processed, embedded, and stored in ChromaDB to enable semantic search and retrieval.

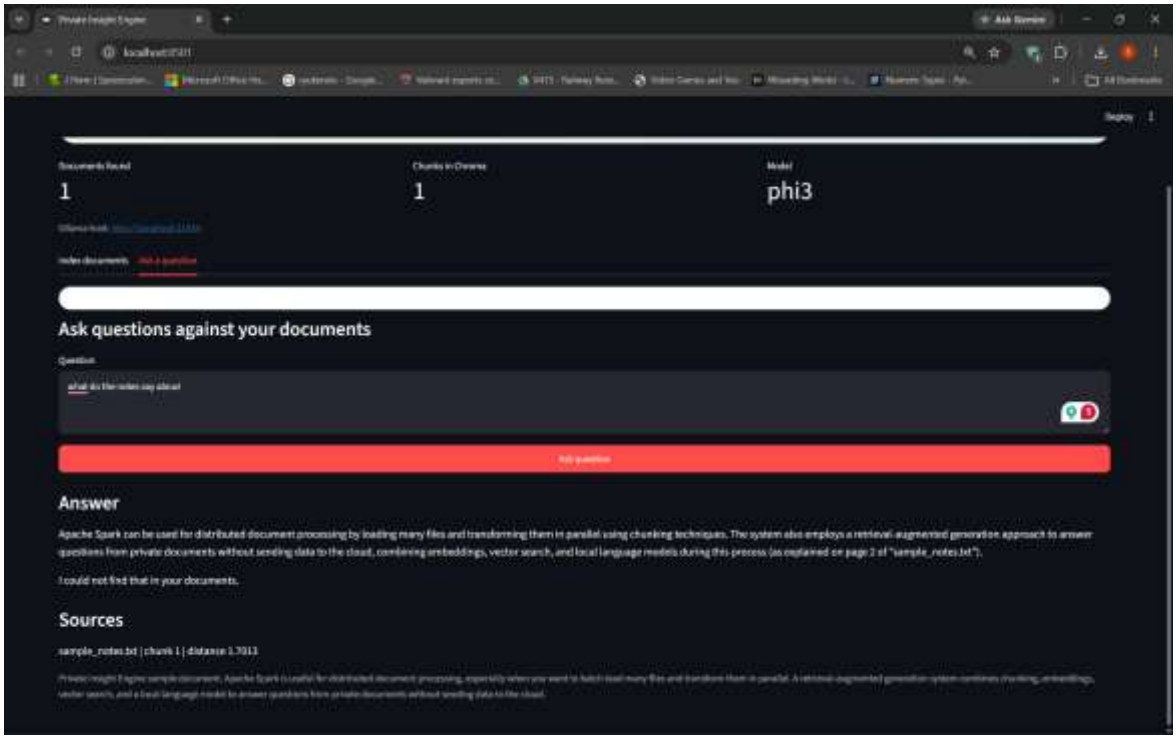


Figure 4.2: Query Processing and Response Generation

The screenshot shows the user submitting a query through the Private Insight Engine interface. The system retrieves relevant document content and generates an answer using the Phi-3 model through Ollama.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1.Conclusion

The ultimate goal of this project was to design and implement a privacy-focused document question-answering system called the Private Insight Engine. The system successfully integrates Apache Spark, Sentence Transformers, ChromaDB, and Ollama to process local documents, perform semantic search, and generate context-aware responses.

The developed system enables users to interact with their documents using natural language while ensuring that all data remains on the local machine. The project demonstrates the practical application of Retrieval-Augmented Generation (RAG) and vector databases in building intelligent and secure document analysis systems.

5.2.FutureWork

In the future, the proposed system can be enhanced further by supporting additional document formats such as DOCX, HTML, and CSV files. Advanced embedding models and retrieval techniques can be incorporated to improve search accuracy and response quality.

The system can also be extended with a web-based interface, conversational memory, multi-user support, and cloud synchronization features. These enhancements would improve usability, scalability, and overall system performance while maintaining data privacy and security.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *arXiv preprint arXiv:2005.11401*, 2020.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv preprint arXiv:2312.10997*, 2023.

[3] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Apache Spark: A Unified Engine for Big Data Processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[4] Apache Software Foundation, “Apache Spark Research,” 2024. Available: https://spark.apache.org/research.html

[5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of EMNLP-IJCNLP*, pp. 3982–3992, 2019.

[6] T. Sato et al., “Evaluating Sentence Embeddings from Diverse Transformer Models,” *Semantic Scholar*, 2024.

[7] P. Henderson et al., “Are There Identifiable Structural Parts in the Sentence Embedding Whole?,” *arXiv preprint arXiv:2406.16563*, 2024.

[8] Chroma Foundation, “Chroma: The AI-Native Vector Database,” 2023. Available: https://www.trychroma.com

[9] Microsoft Research, “Phi-3 Technical Report: A Capable Lightweight Language Model,” *arXiv preprint arXiv:2404.14219*, 2024.

[10] Ollama Labs, “Ollama: Running Large Language Models Locally,” 2024. Available: https://ollama.com

[11] V. Mehta, “I Built a Private AI Document Search Engine That Runs Entirely Locally,” LinkedIn Article, 2025.

[12] Gartner, “Insight Engines Apply Relevancy Methods to Describe, Discover, Organize and Analyze Data,” Sinequa Blog, 2024.